Resilient IP Optimizer

Sandboxes with their own subdomain, a highly available front door, and a platform that cleans up after itself

IEEE CSTAM 3.0 × FelCloud · Phase 2: integration and testing

The problem

Twenty teams arrive at once. Each needs a **VM**, an **address**, a **route** and a **name**.

Later each must vanish without leaving a route, a DNS record or an address behind.

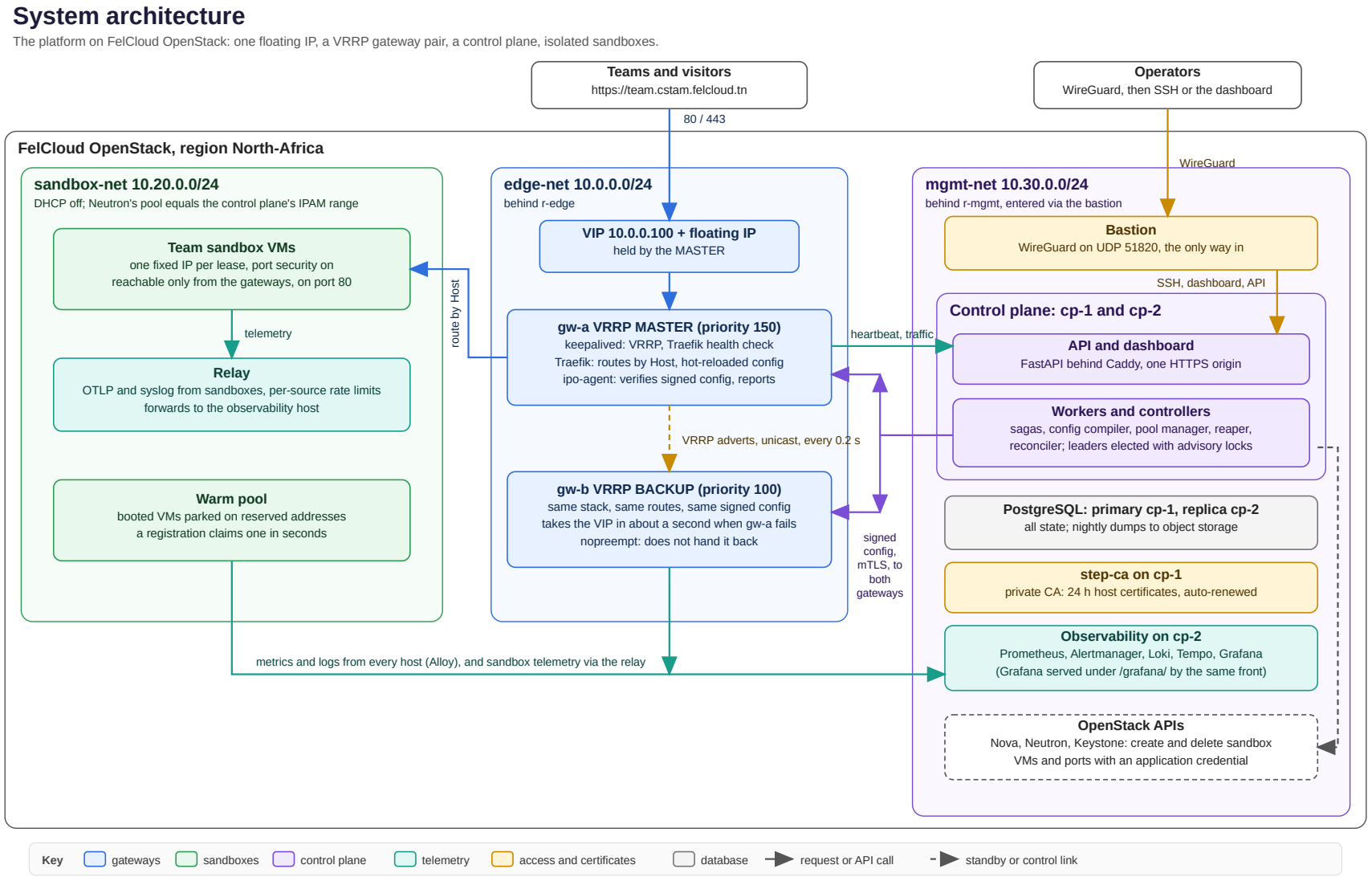
And the front door cannot go down.

100 addresses in the sandbox pool

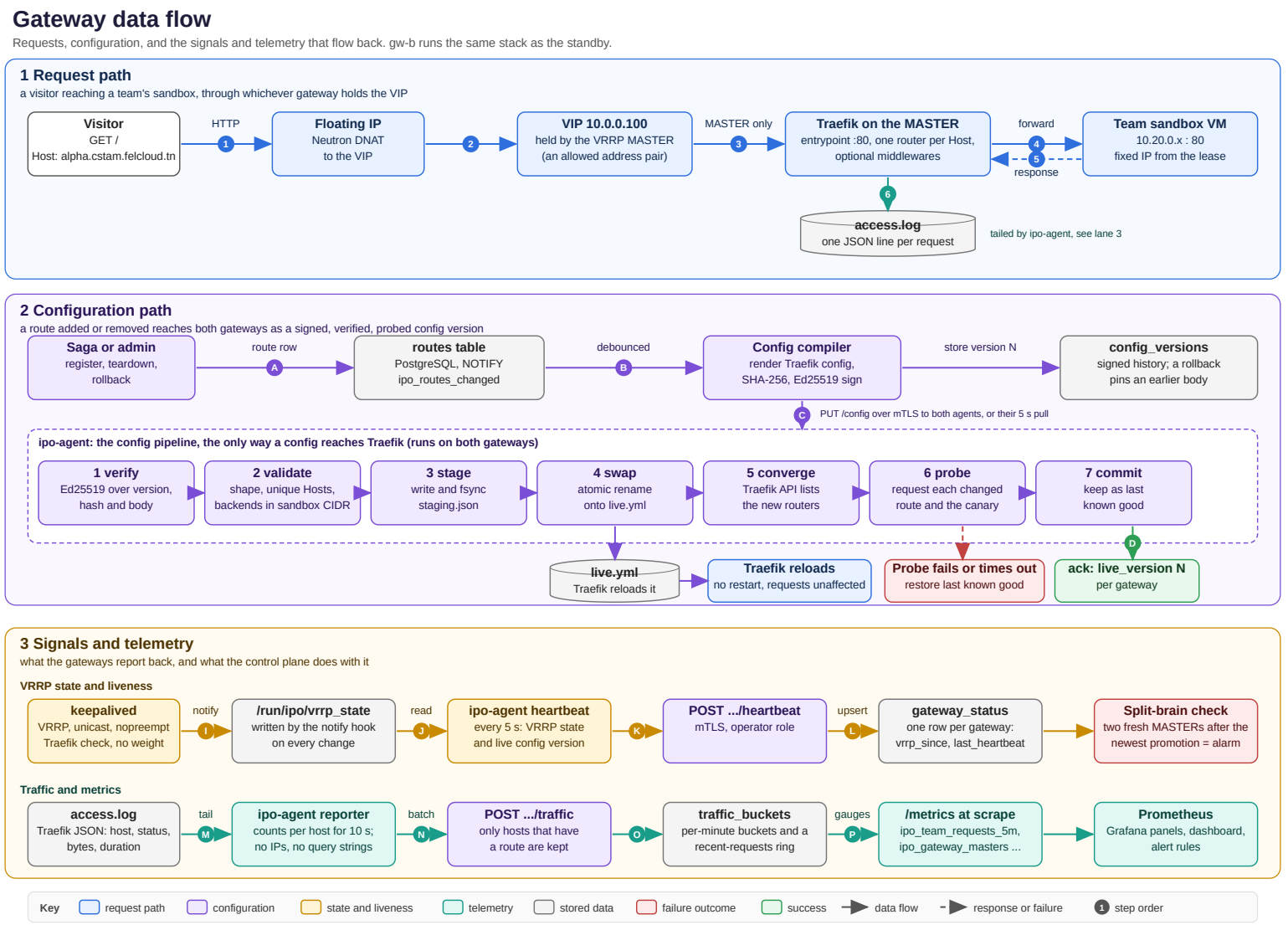
50 instances in the FelCloud quota

17 s to boot one VM, a minute or more when ten boot at once

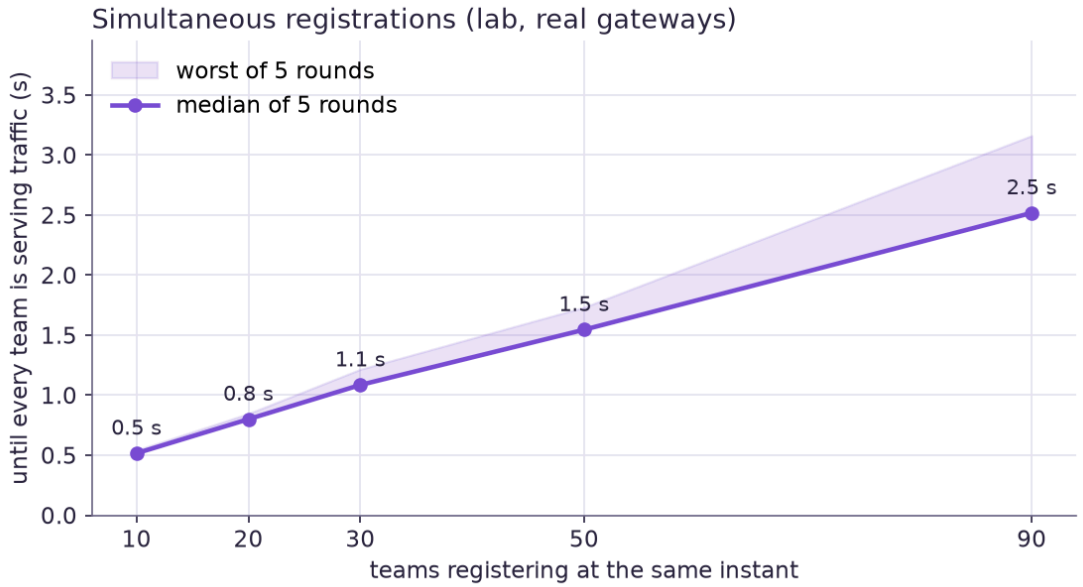
Architecture



Gateway data flow

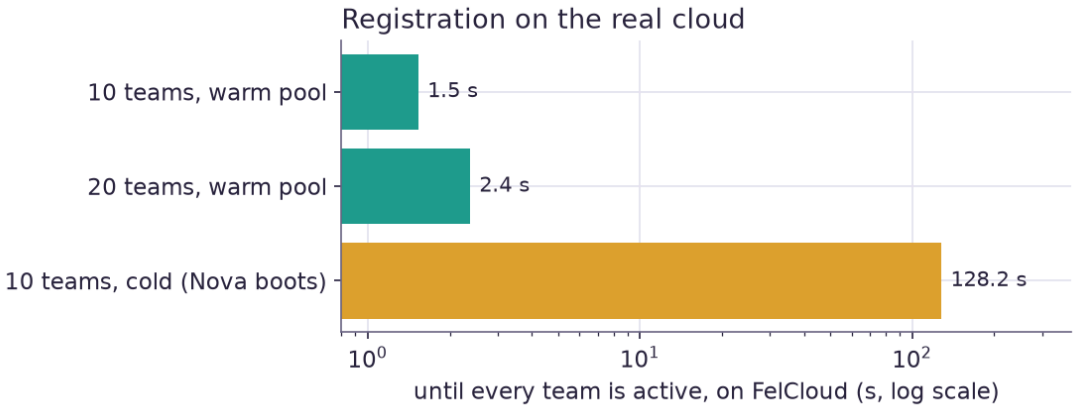


Up to 90 teams at the same instant: no failures



- **1000 registrations** in 5 rounds of 10, 20, 30, 50 and 90 at the same instant: every one **active**, on its **own address**, served by **its own sandbox**; 0 failures.
- One team alone: **0.4 s** from request to serving.
- A saga with idempotency keys, a database-enforced lease state machine, a warm pool that follows demand.
- Measuring found it: 30 teams took **15 s** at first; Traefik's reload throttle and the config batching brought it to **1.1 s**.

On the real FelCloud



- Warm: **20 teams active in 2.4 s**, 0 failures, real Nova and Neutron calls.
- Cold: **108–140 s**. Ten concurrent boots are far slower than one: the warm pool is the lever.
- Teardown of 20 teams: **14 s**; addresses free again **7.5 s** after (5 s quarantine).
- The real cloud caught three bugs the lab could not: warm claims on booting VMs, the reconciler deleting VMs mid-boot, teardown waiting on Nova's lingering DELETED entries (85 s → 6 s).

Deployed for real: the whole platform on FelCloud

- Pulumi built it; **the Ansible roles configured all six VMs**; the sandbox image was baked; then it was used from the internet.
- **A team registered through the real gateways is active 0.6 s after the request** and served from a laptop through the floating IP by its own VM. One whose VM is still booting: active when the VM serves (94 s), first attempt.
- **21 defects no lab could show**, all fixed: FelCloud's routers do not SNAT (the bastion now runs an allow-listed egress proxy); config pushes silently sent no client certificate; the control plane had no path to the VIP for its probe; one slow VM could block every later registration.

6 real VMs configured by Ansible

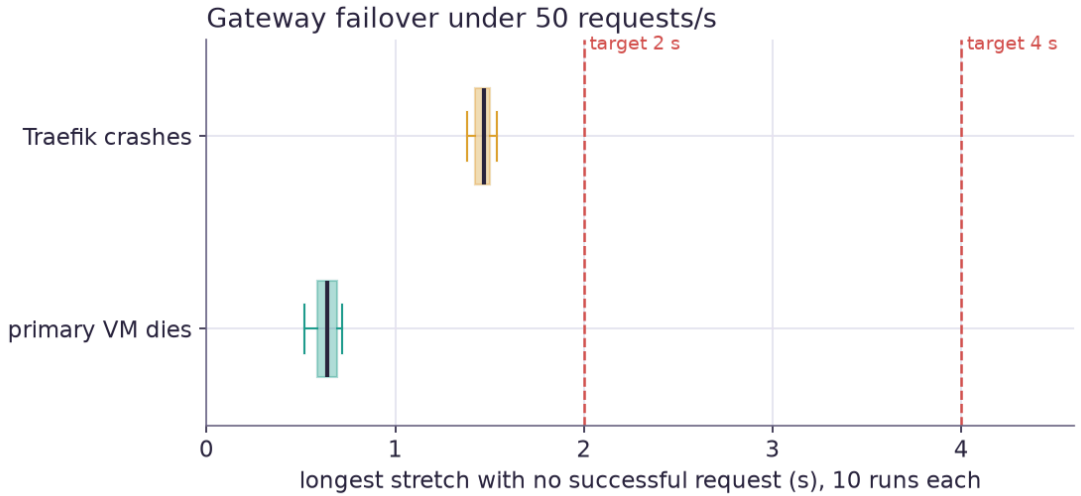
0.6 s registration, real gateways

21 real-world defects fixed

Hot reload and rollback

- Registering or deleting a team changes the routing table **without touching Traefik's process**.
- Config is **Ed25519-signed**; each agent re-validates it (every backend inside the sandbox range), swaps it atomically, probes every changed route, and **rolls back** on failure.
- An operator's rollback reaches both gateways in **0.15 s** (median).
- Control plane killed with `-9` for 8 s under load: **0 failed requests**; back in 1.0 s.
- 2000 requests/s across 30 teams' hosts, new connection each: 0 failures, p99 5.6 ms (lab).

High availability



- VRRP, unicast peers, health check that fails keepalived when Traefik is down.
- **Between two real FelCloud VMs**, through the floating IP: primary loses power **1.0 s**; Traefik down **1.36 s**; planned failover **0.24 s**. The old primary rejoins as BACKUP.
- Lab, many runs: VM dies **0.52–0.72 s**; Traefik **1.4–1.5 s**; planned **0.12 s**.
- Split brain: detected from both heartbeats, alarmed, healed to one master.

Automated teardown: nothing outlives a team

route out	DNS out	VM + port deleted	address verified clean	logins + traffic purged
both gateways ack			then quarantined	

- Route and DNS record gone **2 s** after a lease expires; the address is reusable one quarantine **+2.2 s** later.
- FelCloud: **20 teams torn down in 14 s**.
- Worker killed at every boundary of every step → the reconciler finishes it; 100 create/delete cycles leave zero routes, VMs, ports, DNS records or linked leases. A deleted team's token stops working at once.

Network isolation

- **Three independent layers** from one policy file: Neutron security groups, port security, nftables on every host.
- Lab: **1,782 probes**, each layer alone, forged frames, rerouting, floods.
- **Real FelCloud, probe VMs behind the real security groups:** 270 probes, **0 leaks**. Sandbox→sandbox blocked on every port; gateway management addresses blocked; positive controls pass.
- **Port security proven on the real cloud:** a sandbox sent forged source IPs (a neighbour's, a gateway's, an unused one) and a forged MAC, 32 forged packets: **0 delivered**.

0 leaks of 270 probes

0 of 32 forged packets delivered

3 independent layers

API and security architecture

- **Integration guide:** bearer JWT with roles, idempotent asynchronous operations (`202` + job), RFC 9457 errors, server-sent events, OpenAPI + Postman, DNS modes, client code.
- **STRIDE per component:** edge, gateway agent, API, database, sandboxes, management plane, each threat with its control and the test that would fail without it.
- **Closed:** gateway agents log in with an account bound to their own gateway; every team's route is rate-limited by its Host (a flood gets 429s, its neighbour does not notice); platform egress goes through an allow-listed proxy.
- **Named gaps:** one rate limit for all teams, TLS to teams optional, nftables rather than eBPF.

Evidence

371 control-plane tests on a real PostgreSQL

72 dashboard tests

68 deploy checks on the real config readers

46 + 22 + 35 chaos, load and isolation lab tests

Go agent with `-race`

ruff · mypy --strict · tsc · go vet · ansible-lint clean

Everything above can be re-run: `load.bench`, `chaos.run`, `isolation.run`, `isolation.cloud_probe`, `bench.real_cloud`, and the deployment itself (`site.yml`).

Demo

The scripted demo, recorded against the real lab: subdomain routing → hot reload → a burst of 12 → expiry → failover → VM death → split brain.

`docs/presentation/demo.mp4` (104 s) · `python -m chaos.demo` to run it live (`b` burst, `e` expire, `f` fail over, `k` kill a VM).

Limits and next steps

Not done

- The control plane's real cloud adapter on the deployed hosts (it needs a credential the team issues)
- The observability stack on real hosts; WireGuard with a real peer
- A compute-host failure (soft anti-affinity only)
- Wildcard TLS for team subdomains

Next

- Those, then a rate limit sized per team (today one size for all) and a second region for the control plane